Pseudo code of Priority scheduling non-pre-emptive

```
#include <stdio.h>

int main() {
    int n, i, j;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int Process_ID[n], Arrival_Time[n], Burst_Time[n], Priority[n];
    int Completed[n];
    int Waiting_Time[n], Turnaround_Time[n], Completion_Time[n];

    // Input process details
    for (i = 0; i < n; i++) {
        printf("\nEnter Process ID for process %d: ", i + 1);
        scanf("%d", &Process_ID[i]);
        printf("Enter Arrival Time: ");
        scanf("%d", &Arrival_Time[i]);
        printf("Enter Burst Time: ");
        scanf("%d", &Burst_Time[i]);
        printf("Enter Priority (lower number = higher priority): ");
        scanf("%d", &Priority[i]);
        Completed[i] = 0; // Initialize as not completed
    }

    int Current_Time = 0;
    int Completed_Count = 0;
```

```
while (Completed_Count < n) {
    int Highest_Priority = 100000; // Large number
    int Selected_Process = -1;

    // Find process with highest priority that has arrived
    for (i = 0; i < n; i++) {
        if (Arrival_Time[i] <= Current_Time && Completed[i] == 0) {
            if (Priority[i] < Highest_Priority) {
                Highest_Priority = Priority[i];
                Selected_Process = i;
            }
            // If priority is same, choose earlier arrival
            else if (Priority[i] == Highest_Priority) {
                if (Arrival_Time[i] < Arrival_Time[Selected_Process]) {
                    Selected_Process = i;
                }
            }
        }
    }

    if (Selected_Process == -1) {
        Current_Time++; // CPU idle
    } else {
        int Start_Time = Current_Time;
        Completion_Time[Selected_Process] = Start_Time + Burst_Time[Selected_Process];
        Turnaround_Time[Selected_Process] = Completion_Time[Selected_Process] -
Arrival_Time[Selected_Process];
```

```
        Waiting_Time[Selected_Process] = Turnaround_Time[Selected_Process] -
Burst_Time[Selected_Process];


        Current_Time = Completion_Time[Selected_Process];
        Completed[Selected_Process] = 1;
        Completed_Count++;
    }
  }

  // Print results
  printf("\nProcessID\tArrival\tBurst\tPriority\tWaiting\tTurnaround\n");
  for (i = 0; i < n; i++) {
    printf("%d\t\t%d\t%d\t%d\t\t%d\t%d\n",
          Process_ID[i],
          Arrival_Time[i],
          Burst_Time[i],
          Priority[i],
          Waiting_Time[i],
          Turnaround_Time[i]);
  }

  // Optional: Average WT and TAT
  float totalWT = 0, totalTAT = 0;
  for (i = 0; i < n; i++) {
    totalWT += Waiting_Time[i];
    totalTAT += Turnaround_Time[i];
  }
  printf("\nAverage Waiting Time: %.2f", totalWT / n);
```

```
    printf("\nAverage Turnaround Time: %.2f\n", totalTAT / n);


    return 0;
}
```

OUTPUT:

```
"C:\Users\Admin\Desktop\1W

Enter Burst Time: 5
Enter Priority (lower number = higher priority): 2

Enter Process ID for process 2: 2
Enter Arrival Time: 1
Enter Burst Time: 3
Enter Priority (lower number = higher priority): 1

Enter Process ID for process 3: 3
Enter Arrival Time: 2
Enter Burst Time: 8
Enter Priority (lower number = higher priority): 4

Enter Process ID for process 4: 4
Enter Arrival Time: 3
Enter Burst Time: 6
Enter Priority (lower number = higher priority): 2

ProcessID       Arrival Burst   Priority        Waiting Turnaround
1               0       5       2               0       5
2               1       3       1               4       7
3               2       8       4               12      20
4               3       6       2               5       11

Average Waiting Time: 5.25
Average Turnaround Time: 10.75

Process returned 0 (0x0)   execution time : 32.659 s
Press any key to continue.
```

Pseudo code of Priority scheduling Pre-emptive

```
#include <stdio.h>


int main() {

    int n, i, j;
```

```
printf("Enter number of processes: ");
scanf("%d", &n);

int Process_ID[n], Arrival_Time[n], Burst_Time[n], Priority[n];
int Remaining_Time[n];
int Completion_Time[n], Waiting_Time[n], Turnaround_Time[n];

// Input process details
for(i = 0; i < n; i++) {
    printf("\nEnter Process ID for process %d: ", i + 1);
    scanf("%d", &Process_ID[i]);
    printf("Enter Arrival Time: ");
    scanf("%d", &Arrival_Time[i]);
    printf("Enter Burst Time: ");
    scanf("%d", &Burst_Time[i]);
    printf("Enter Priority (lower number = higher priority): ");
    scanf("%d", &Priority[i]);
    Remaining_Time[i] = Burst_Time[i]; // Initialize remaining time
}

int Current_Time = 0, Completed = 0;

while(Completed < n) {
    int Highest_Priority = 100000; // Large number
    int Selected_Process = -1;

    // Find highest priority process at current time
```

```
for(i = 0; i < n; i++) {
    if(Arrival_Time[i] <= Current_Time && Remaining_Time[i] > 0) {
        if(Priority[i] < Highest_Priority) {
            Highest_Priority = Priority[i];
            Selected_Process = i;
        }
        // If priority is same, choose earlier arrival
        else if(Priority[i] == Highest_Priority) {
            if(Arrival_Time[i] < Arrival_Time[Selected_Process]) {
                Selected_Process = i;
            }
        }
    }
}

if(Selected_Process == -1) {
    Current_Time++; // CPU idle
} else {
    // Execute process for 1 time unit
    Remaining_Time[Selected_Process]--;
    Current_Time++;

    // If process finished
    if(Remaining_Time[Selected_Process] == 0) {
        Completion_Time[Selected_Process] = Current_Time;
        Turnaround_Time[Selected_Process] = Completion_Time[Selected_Process] -
Arrival_Time[Selected_Process];
        Waiting_Time[Selected_Process] = Turnaround_Time[Selected_Process] -
Burst_Time[Selected_Process];
```

```
            Completed++;
        }
    }
}

// Print results
printf("\nProcessID\tArrival\tBurst\tPriority\tWaiting\tTurnaround\n");
for(i = 0; i < n; i++) {
    printf("%d\t\t%d\t%d\t%d\t\t%d\t%d\n",
            Process_ID[i],
            Arrival_Time[i],
            Burst_Time[i],
            Priority[i],
            Waiting_Time[i],
            Turnaround_Time[i]);
}

// Optional: Average WT and TAT
float totalWT = 0, totalTAT = 0;
for(i = 0; i < n; i++) {
    totalWT += Waiting_Time[i];
    totalTAT += Turnaround_Time[i];
}
printf("\nAverage Waiting Time: %.2f", totalWT / n);
printf("\nAverage Turnaround Time: %.2f\n", totalTAT / n);

return 0;
}
```

OUTPUT

```
"C:\Users\Admin\Desktop\1W

Enter Burst Time: 5
Enter Priority (lower number = higher priority): 2

Enter Process ID for process 2: 2
Enter Arrival Time: 1
Enter Burst Time: 3
Enter Priority (lower number = higher priority): 1

Enter Process ID for process 3: 3
Enter Arrival Time: 2
Enter Burst Time: 8
Enter Priority (lower number = higher priority): 4

Enter Process ID for process 4: 4
Enter Arrival Time: 3
Enter Burst Time: 6
Enter Priority (lower number = higher priority): 2

ProcessID       Arrival Burst   Priority        Waiting Turnaround
1               0       5       2               0       5
2               1       3       1               4       7
3               2       8       4               12      20
4               3       6       2               5       11

Average Waiting Time: 5.25
Average Turnaround Time: 10.75

Process returned 0 (0x0)   execution time : 32.659 s
Press any key to continue.
```

Round Robin

// Round Robin Scheduling

```
#include <stdio.h>


int main() {

   int n, time_quantum, i, j;


   printf("Enter number of processes: ");

   scanf("%d", &n);
```

```
printf("Enter Time Quantum: ");
scanf("%d", &time_quantum);

int process_id[n], arrival_time[n], burst_time[n], remaining_time[n];
int waiting_time[n], turnaround_time[n];

// Input process details
for(i = 0; i < n; i++) {
    printf("\nEnter Process ID for process %d: ", i + 1);
    scanf("%d", &process_id[i]);
    printf("Enter Arrival Time: ");
    scanf("%d", &arrival_time[i]);
    printf("Enter Burst Time: ");
    scanf("%d", &burst_time[i]);
    remaining_time[i] = burst_time[i]; // initialize remaining time
    waiting_time[i] = 0;
    turnaround_time[i] = 0;
}

int current_time = 0, completed = 0;
int ready_queue[1000], front = 0, rear = 0;
int in_queue[n]; // to mark if process is in queue
for(i = 0; i < n; i++) in_queue[i] = 0;

while(completed < n) {
    // Add all arrived processes not yet completed and not in queue
    for(i = 0; i < n; i++) {
```

```
    if(arrival_time[i] <= current_time && remaining_time[i] > 0 && in_queue[i] == 0) {
        ready_queue[rear++] = i;
        in_queue[i] = 1;
    }
}

if(front == rear) { // ready queue empty → CPU idle
    current_time++;
    continue;
}

int p = ready_queue[front++]; // select process at front

if(remaining_time[p] > time_quantum) {
    remaining_time[p] -= time_quantum;
    current_time += time_quantum;

    // Add newly arrived processes during this time
    for(i = 0; i < n; i++) {
        if(arrival_time[i] <= current_time && remaining_time[i] > 0 && in_queue[i] == 0) {
            ready_queue[rear++] = i;
            in_queue[i] = 1;
        }
    }

    ready_queue[rear++] = p; // put back at end of queue
} else {
    current_time += remaining_time[p];
```

```
            turnaround_time[p] = current_time - arrival_time[p];
            waiting_time[p] = turnaround_time[p] - burst_time[p];
            remaining_time[p] = 0;
            completed++;
        }
    }

    // Print results
    printf("\nProcessID\tArrival\tBurst\tWaiting\tTurnaround\n");
    for(i = 0; i < n; i++) {
        printf("%d\t\t%d\t%d\t%d\t%d\n",
                process_id[i], arrival_time[i], burst_time[i], waiting_time[i], turnaround_time[i]);
    }

    // Calculate averages
    float avgWT = 0, avgTAT = 0;
    for(i = 0; i < n; i++) {
        avgWT += waiting_time[i];
        avgTAT += turnaround_time[i];
    }
    printf("\nAverage Waiting Time: %.2f", avgWT / n);
    printf("\nAverage Turnaround Time: %.2f\n", avgTAT / n);

    return 0;
}
```

OUTPUT

"C:\Users\Admin\Desktop\1W

```
Enter Time Quantum: 3

Enter Process ID for process 1: 1
Enter Arrival Time: 0
Enter Burst Time: 5

Enter Process ID for process 2: 2
Enter Arrival Time: 1
Enter Burst Time: 3

Enter Process ID for process 3: 3
Enter Arrival Time: 2
Enter Burst Time: 8

Enter Process ID for process 4: 4
Enter Arrival Time: 3
Enter Burst Time: 6

ProcessID       Arrival Burst   Waiting Turnaround
1               0       5       9       14
2               1       3       2       5
3               2       8       12      20
4               3       6       11      17

Average Waiting Time: 8.50
Average Turnaround Time: 14.00

Process returned 0 (0x0)   execution time : 30.536 s
Press any key to continue.
```